

Asymetrické šifrování

Asymetrické šifrování řeší problém distribuce klíče k příjemci. Zde, na rozdíl od symetrického šifrování, není potřeba dostat tajný klíč bezpečně k druhé straně. Tento způsob šifrování používá dva typy klíčů. Jeden klíč je určen pro šifrování citlivých dat a druhý klíč je určen pro jejich dešifrování. Tyto dva klíče si uživatel vygeneruje zároveň a stává se tak jejich jediným majitelem. Generování těchto klíčů je založeno na astronomicky obrovských prvočíslech. Klíče jsou tedy velmi velká čísla. Jsou měřena na počet bitů (např. 1024 bitů).

Je zřejmé, že šifrovací klíč **e** a dešifrovací klíč **d** spolu musí být matematicky svázány, avšak nezbytnou podmínkou pro užitečnost šifry je praktická nemožnost ze znalosti šifrovacího klíče spočítat dešifrovací a naopak. Matematicky tedy asymetrická kryptografie postupuje následujícím způsobem:

Šifrování

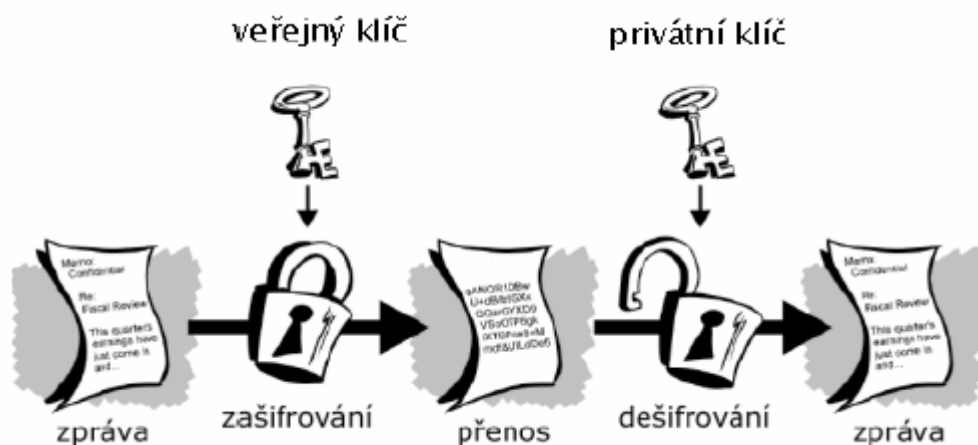
Dešifrování

$$c = f(m, e)$$

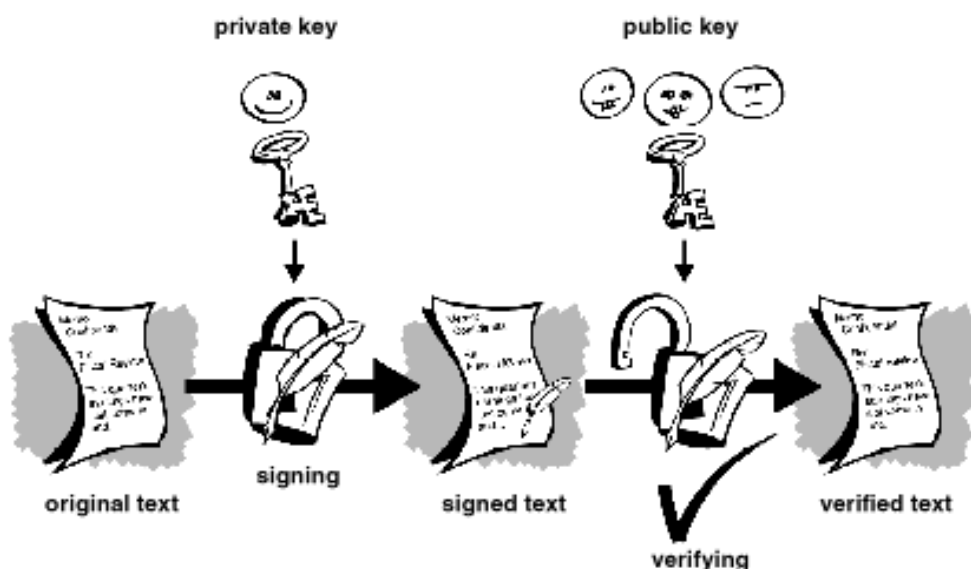
$$m = g(c, d)$$

V principu se mohou šifrovací a dešifrovací funkce lišit, zpravidla jsou však matematicky přinejmenším velmi podobné.

Princip asymetrického šifrování je následující: jeden klíč si uživatel vybere jako veřejný a zpřístupní jej ostatním. Poskytnutí tohoto veřejného klíče může uživatel realizovat buď zpřístupněním na svém webu, poslat jej může elektronickou poštou, dát k dispozici certifikační autoritě, nebo jiným možným způsobem. Druhý z dvojce klíčů si uživatel dobře schová a nesmí jej poskytnout nikomu jinému, jinak by šifrování nebylo účinné. Pokud uživatel chce poslat nějaká citlivá data, zašifruje tyto data veřejným klíčem příjemce, který předem získá. Takto vznikne zašifrovaný text, který neumí nikdo jiný dešifrovat v rozumném čase než příjemce. Kdokoliv jiný se může pokusit zprávu dešifrovat jiným privátním klíčem, ale protože nepatří jako pár ke klíči ve kterém byla informace zašifrována dostane pouze nějaký shluk nečitelných dat. Příjemce tedy jako jediný může zprávu dešifrovat a získá tak původní data před jejich zašifrováním. Následující obrázek naznačuje předešlý popsany postup šifrování.

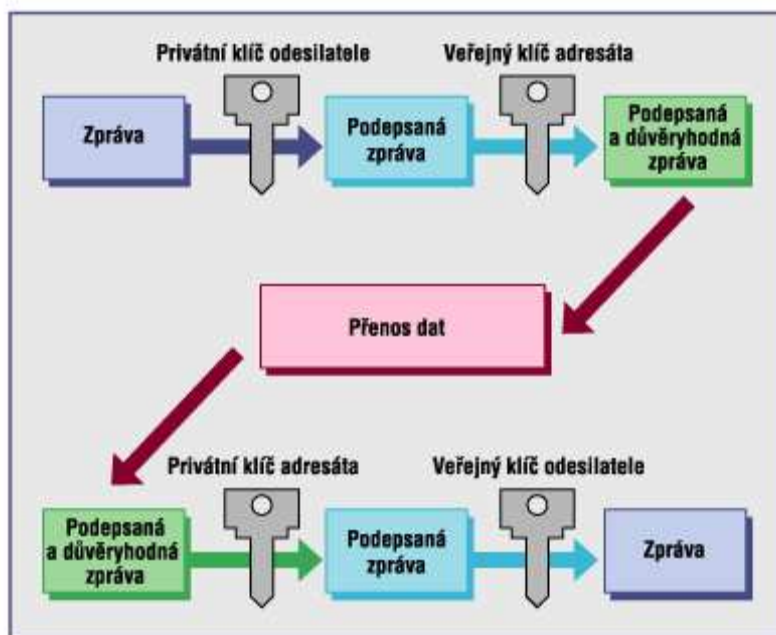


Postup podepsání dokumentu je podobný jako v předešlém případě. Uživatel zašifruje svá data svým privátním klíčem. Takto vznikne zašifrovaný text, který může kdokoli jiný dešifrovat. Příjemce tedy dešifruje zprávu veřejným klíčem vysílače a získá tak původní data, která jsou podepsána (příjemce je vlastníkem a autorem dat). Obrázek naznačuje postup podepsání zprávy.



Kombinací obou předešlých případů, tedy podepsání a bezpečnému zašifrování lze dosáhnout přenosu zprávy jak důvěryhodné (zabezpečené) tak i autorizované (podepsané). Tedy vysílač zašifruje zprávu svým privátním klíčem (zpráva podepsaná) a následně tuto zprávu zašifruje veřejným klíčem příjemce (podepsaná a zabezpečená zpráva). Nikdo jiný kromě příjemce nemůže v rozumném čase zprávu dešifrovat ani změnit odesilatele, jelikož opět chybí klíč k podepsání zprávy. Příjemce tedy nejprve zprávu dešifruje svým privátním klíčem, tím dostane podepsanou

zprávu, kterou dešifruje veřejným klíčem odesílatele a získá tak původní citlivá data. Postup je znázorněn na dalším obrázku:



RSA

RSA (iniciály příjmení autorů: Ron Rivest, Adi Shamir, Len Adleman) je asymetrická šifra s veřejným klíčem, jedná se o první algoritmus, který je vhodný jak pro podepisování, tak šifrování. Publikován byl v roce 1977 a používá se i dnes, přičemž při dostatečné délce klíče je považován za bezpečný. Algoritmus byl v roce 1983 patentován a vypršel v roce 2000.

RSA algoritmus používá Fermatovu větu a modulární aritmetiku. Síla této šifry spočívá v tom, že dosud nebyla objevena metoda, jak rozložit velká čísla na prvočísla. Zatím není ani zcela jisté, zda je vůbec možné takovou metodu objevit. Pokud se tak ovšem stane, bude tato šifra nepoužitelná.

Popis činnosti algoritmu

Bezpečnost RSA je postavena na předpokladu, že rozložit velké číslo na součin prvočísel (faktorizace) je velmi obtížná úloha. Z čísla $n = pq$ je tedy v rozumném čase prakticky nemožné zjistit činitele p a q , neboť není znám žádný algoritmus

faktorizace, který by pracoval v polynomiálním čase vůči velikosti binárního zápisu čísla n . Naproti tomu násobení dvou velkých čísel je elementární úloha.

Příklad:

Alice a Bob chtějí komunikovat prostřednictvím otevřeného (nezabezpečeného) kanálu a Bob by chtěl Alici poslat soukromou zprávu.

Nejprve si bude Alice muset vyrobit pár veřejného a soukromého klíče:

1. Zvolí dvě různá velká náhodná prvočísla p a q .
2. Spočítá jejich součin $n = pq$.
3. Spočítá hodnotu Eulerovy funkce $\varphi(n) = (p - 1)(q - 1)$.
4. Zvolí celé číslo e menší než $\varphi(n)$, které je s $\varphi(n)$ nesoudělné.
5. Nalezne číslo d tak, aby platilo $e \cdot d \equiv 1 \pmod{\varphi(n)}$.
6. Jestli e je prvočíslo tak $d = (1 + r \cdot \varphi(n)) / e$, kde $r = [(e-1)\varphi(n)^{(e-2)}]$

Veřejným klíčem je dvojice (n, e) , přičemž n se označuje jako modul, e jako šifrovací či veřejný exponent. Soukromým klíčem je dvojice (n, d) , kde d se označuje jako dešifrovací či soukromý exponent. (V praxi se klíče uchovávají v mírně upravené formě, která umožňuje rychlejší zpracování.)

Veřejný klíč poté Alice uveřejní, respektive zcela nepokrytě pošle Bobovi. Soukromý klíč naopak uchová v tajnosti.

Zašifrování zprávy

Bob nyní chce Alici zaslat zprávu M . Tuto zprávu převede nějakým dohodnutým postupem na číslo m ($m < n$). Šifrovým textem odpovídajícím této zprávě pak je číslo

$$c = m^e \bmod n$$

Tento šifrový text poté zašle nezabezpečeným kanálem Alici.

Dešifrování zprávy

Alice od Boba získá šifrový text c . Původní zprávu m získá následujícím výpočtem:

$$m = c^d \bmod n$$

Fakt, že tímto výpočtem získáme původní zprávu, je důsledkem následující rovnosti:

$$c^d \equiv (m^e)^d \equiv m^{ed} \pmod{n}$$

A jelikož $ed \equiv 1 \pmod{p-1}$ a $ed \equiv 1 \pmod{q-1}$, díky malé Fermatově větě platí, že $m^{ed} \equiv m \pmod{p}$ a zároveň $m^{ed} \equiv m \pmod{q}$

Jelikož p a q jsou různá prvočísla, pomocí čínské věty o zbytcích je dáno

$$m^{ed} \equiv m \pmod{pq}$$

Tudíž $c^d \equiv m \bmod n$.

Příklad

V tomto příkladu jsou pro jednoduchost použita extrémně malá čísla, v praxi se používají o mnoho řádů větší.

$$p = 61 \text{ (první prvočíslo)}$$

$$q = 53 \text{ (druhé prvočíslo)}$$

$$n = pq = 3233 \text{ (modul, veřejný)}$$

$$e = 17 \text{ (veřejný, šifrovací exponent)}$$

$$d = 2753 \text{ (soukromý, dešifrovací exponent)}$$

Pro zašifrování zprávy **123** probíhá výpočet:

$$\text{šifruj}(123) = 123^{17} \bmod 3233 = 855$$

Pro dešifrování pak:

$$\text{dešifruj}(855) = 855^{2753} \bmod 3233 = 123$$

Digitální podpis

Algoritmus RSA lze snadno využít pro digitální podpis. Základním principem takového využití je „opačné“ použití šifry – pokud Alice chce poslat Bobovi podepsanou zprávu, připojí k ní číslo získané jakoby „dešifrováním“ haše své zprávy pomocí svého soukromého klíče. Bob poté jakoby zpětně „zašifruje“ tento podpis pomocí Alicina veřejného klíče a porovná výsledek s hašem zprávy. Pokud zpráva nebyla změněna, vyjde stejná hodnota, neboť algoritmus je z hlediska šifrování i dešifrování symetrický (lze zaměnit e a d). Jelikož jediný, kdo zná tajný klíč Alice, je Alice, je tím zaručeno, že ho zašifrovala Alice.

Vyplňovací schémata

V praxi se musí RSA kombinovat s nějakým typem tzv. vyplňovacího schématu, jelikož hodnoty m způsobují ztrátu bezpečnosti šifrovaného textu. RSA použité bez vyplňovacího schématu může trpět množstvím potíží:

- Hodnoty $m = 0$ nebo $m = 1$ vždy produkují šifrovaný text rovný 0 nebo 1.
- Když je použito šifrování s malým exponentem (např. $e = 3$) a malé hodnoty m , výsledek m^e může být podstatně nižší než n . V tomto případě šifrované texty mohou být snadno dešifrovány použitím e - kořenu šifrovaného textu bez nutnosti vyžadování znalosti modulu.
- Jelikož je RSA deterministický šifrovací algoritmus, nemá žádné proměnné komponenty. Útočník může snadno použít útok nešifrovaným textem za pomoci slovníku a porovnáváním jednotlivých částí šifrovaného textu s ním.

V praxi mohou první dva problémy nastat hlavně při posílání krátkých ASCII zpráv, kde m je řetězec jednoho nebo více ASCII znaků. Zpráva obsahující ASCII znak NUL (kterého numerická hodnota je 0) může být dekodována jako $m = 0$, což produkuje šifrovaný text jako nuly, bez ohledu na to, jaké e a N je použito. Rovněž ASCII znak SOH (numerická hodnota je 1) vždy vyprodukuje šifrovaný text jedniček. V systémech, které používají malé hodnoty e jako třeba 3, mohou být všechny ASCII znakové zprávy málo bezpečné, jelikož m může nabývat nejvýše hodnoty 255 a 2553 je méně než snesitelný modul. Takové texty mohou být obnoveny jednoduše použitím umocňování a odmocňování.

K zabránění těchto problémů implementace RSA obvykle skrývají některý typ náhodného vyplnění hodnoty m před jejím zašifrováním. Vyplnění způsobí, že m nemůže klesnout na nebezpečnou hodnotu a takto získaná „zaplněná“ zpráva bude zašifrována do bezpečné hodnoty šifrovaného textu. Navíc to pomůže snížit možnost útoku pomocí slovníkových metod.

Standardy PKCS (Public-Key Cryptography Standards) byly vytvořeny k bezpečnému vyplnění zpráv před jejich zašifrováním pomocí RSA. Jelikož tato schémata zaplní nešifrovaný text m určitým počtem dodatečných znaků, musí být velikost nevyplněné zprávy M o něco menší. RSA vyplňovací schémata musí být opatrně navržena k tomu, aby zabránila i velmi sofistikovaným útokům usnadněným rovněž předvídatelnou strukturou zprávy. První verze PKCS používaly ad-hoc strukturu, která se později ukázala jako nedostatečně účinná proti útoku pomocí srovnávání šifrovaného textu se slovníkem. Moderní konstrukce používají bezpečnostní techniky jako Optimální Asymetrické Šifrovací Vyplňování (OAEP) k ochraně zpráv proti tomuto typu útoku. PKCS standard rovněž začleňuje zpracovávací schémata k poskytnutí bezpečnosti pro RSA podpisy, (např. RSA-PSS).

Bezpečnost

Bezpečnost šifrovacího systému RSA je založena na dvou matematických problémech: problém faktorizace velmi vysokých čísel a RSA problému. Plné dešifrování RSA šifrovaného textu je obtížné v podstatě neproveditelné, jelikož neznáme algoritmus, pomocí nějž by se to dalo provést. Odolnost vůči částečnému dešifrování zprávy může vyžadovat použití některého ze známých způsobů vyplňování.

RSA problém je definován jako úloha získání e -tého kořenu modulu množiny n , obnovení hodnoty m jako $m^e = c \bmod n$, kde (e, n) jsou RSA veřejné klíče a c je RSA šifrovaný text. V současnosti nejslibnější přístup k vyřešení RSA problému je faktor modulu n . Se schopností obnovit primární faktory může útočník spočítat tajný exponent d z veřejného klíče (e, n) , a pak dešifrovat c použitím standardního postupu. Útočník tohoto dosáhne faktorem n do p a q , a vypočítá $(p-1)(q-1)$ a to umožní stanovení d pro e . Nebyla nalezena polynomiálně časově složitá metoda pro faktory celých velkých čísel na klasických počítačích, ale nebylo dokázáno, že by to nešlo.

V roce 2005, největší číslo faktoru univerzálními metodami bylo 663 bitů dlouhé, použitím distribučních metod. RSA klíče jsou typicky dlouhé 1024 – 2048 bitů. Někteří experti věří, že klíče 1024 bitů se mohou v blízké době stát prolomitelnými (ačkoli toto je sporné), ale není žádná známa cesta která by mohla v nejbližší budoucnosti prolomit klíč 4096 bitů dlouhý. Proto, se obecně předpokládá, že RSA je bezpečný, jestliže n je dostatečně velké. Jestliže n je 256 bitů nebo kratší, může být za pár hodin faktorizován na osobním počítači, za použití volně dostupného softwaru. Jestliže n je 512 bitů nebo kratší, může být faktorizován několika sty počítači. Teoretické hardwarové zařízení pojmenované TWIRL a popsané Shamirem a Tromerem v roce 2003 vyvolalo otázku na bezpečnost 1024 bitového klíče. V současné době je doporučováno, aby n bylo alespoň 2048 bitů dlouhé.

V roce 1993 publikoval Peter Shor svůj algoritmus, který ukazoval, že by kvantový počítač mohl v principu vykonávat faktorizaci v polynomiálním čase, což by učinilo RSA a příbuzné algoritmy zastaralými. Realizace principů kvantového počítání se však v současnosti potýká s takovými praktickými problémy, že se o bezpečnost zašifrovaných dat zatím není třeba bát.

V roce 1995 byl popsán časový útok na RSA – jestliže útočník dokáže měřit dešifrovací časy na několika známých šifrovaných textech, může útočník odhalit klíč k dešifrování d .

V roce 1998 David Bleichenbacher popsal útok pomocí mycího se textu – v důsledku této práce byly upraveny optimální šifrovací výplně a RSA laboratoř vydala novou verzi PKCS #1.

Generování klíčů

K nalezení velkých prvočísel p a q obvykle vezmeme náhodná čísla správné velikosti a to, zda jsou s velkou pravděpodobností prvočísla, otestujeme za pomoci testu prvočíselnosti.

Hodnoty p a q by neměly být příliš blízká, aby Fermatova faktorizace pro n nebyla úspěšná. Dále jestliže $p-1$ nebo $q-1$ se rozkládá pouze na malá prvočísla, n může být rychle faktorizováno, a proto je nutné takové hodnoty nepoužít.

Pro generování p a q potřebujeme dobrý generátor náhodných čísel, jehož chování nebude předvídatelné. Všimněme si obou požadavků 'náhodný' a 'nepředvídatelný'. Tato kritéria nejsou totožná; číslo může být vybráno náhodným procesem (žádný model výsledku), ale i to je nějakým způsobem předvídatelné, použitá metoda bude mít za následek ztrátu bezpečnosti. Například v roce 1950 byl publikován přehled čísel, která by mohla být dostatečně náhodná, ale právě tato publikace může sloužit jako návod k útokům. Jestliže útočník může odhadnout polovinu čísel p nebo q , může rychle vypočítat druhou polovinu (to ukázal Coppersmith v r. 1997).

Je důležité, aby tajný klíč d byl dost velký. Wiener ukázal v roce 1990, že pokud $q < p < 2q$ (obvyklé) a $d < n^{1/4}/3$, pak d může být účinně vypočítáno z n a e . Není žádný známý útok proti malému veřejnému exponentu jako například $e=3$, za předpokladu, že je použita správná výplň. Avšak, když žádná výplň použitá není nebo když je výplň nevhodná, potom je větší riziko útoku, jako například rozbalit bezbranný holý text. Běžně používaná hodnota pro e je 65537. Tato hodnota může být považována za kompromis mezi potenciálními útoky na malý exponent a stále účinnému šifrování (či podpisovému ověřování). NIST návrh z března 2006 nedovoluje dělat veřejné exponenty e menší než 65537, ale neuvádí důvod pro toto omezení.

Distribuce klíčů

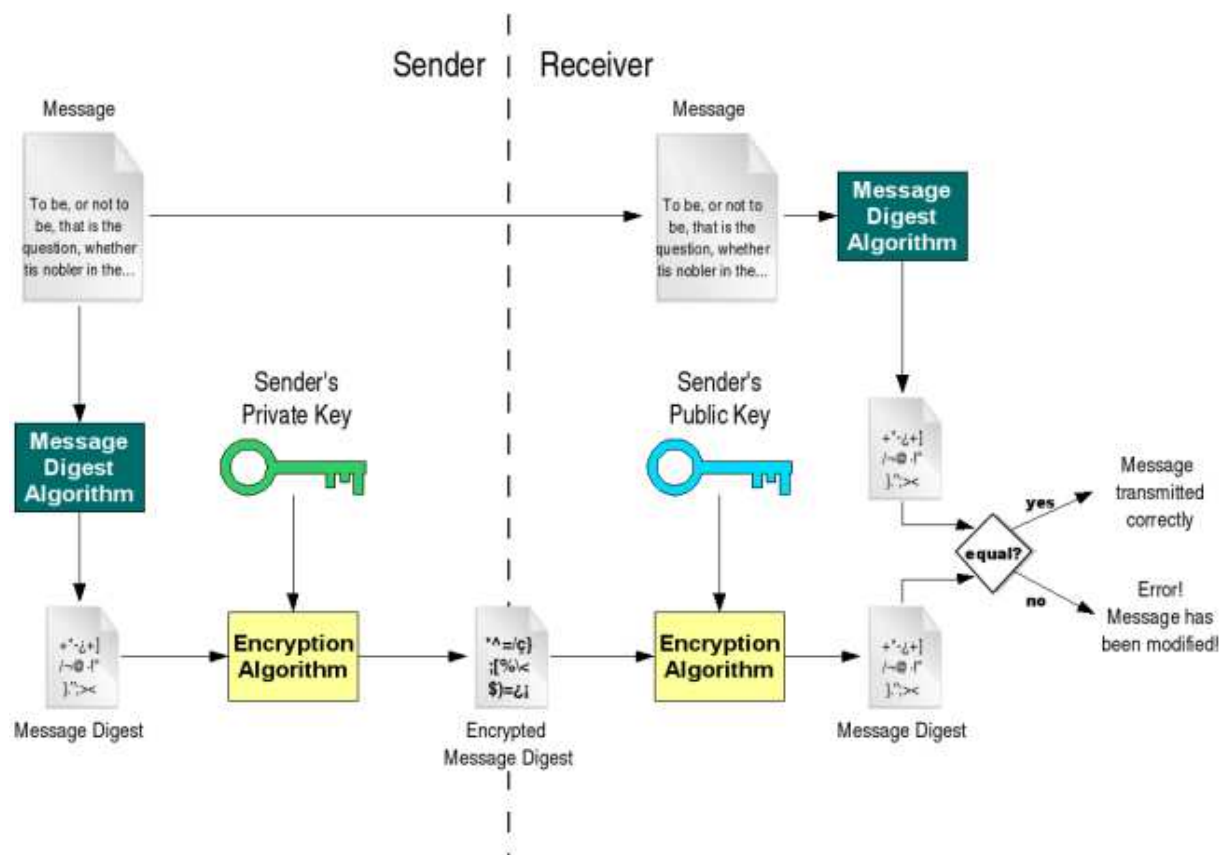
Jako u všech šifer, je i zde důležité, jak si uživatelé předají klíče. Distribuce klíčů musí být zabezpečena před třetí osobou (man-in-the-middle attack). Dejme tomu, že Eva (třetí osoba) zná nějaký způsob jak podstrčit Bobovi svůj klíč a navléknout vše tak, aby si Bob myslel, že tento klíč patří Alici. Předpokládejme dále, že Eva je napíchnutá na přenosy mezi Alicí a Bobem. Eva pošle Bobovi svůj veřejný klíč, Bob věří, že jde o Alici. Eva pak může zachytit jakýkoliv šifrovaný text poslaný Bobem, dešifrovat zprávu podstrčeným klíčem, získat kopii zprávy, zašifrovat zprávu veřejným klíčem Alice, a poslat nový šifrovaný text k Alici. V principu, Alice ani Bob by nebyli schopni odhalit přítomnost Evy. Obrana proti takovým útokům je často založena na elektronickém podpisu.

DSA

Digital Signature Algorithm je standard americké vlády pro digitální podpis. Byl navržen americkým institutem NIST v srpnu 1991 pro použití v jejich Digital Signature Standard (DSS), jenž byl přijat v roce 1993.

DSA je patentovaný a připsaný Davidovi W. Kravitzovi, bývalému zaměstnanci Národní bezpečnostní agentury. Národní institut standardů a technologie tento patent dal celosvětové veřejnosti k volnému užívání bez poplatků.

Vysílač před odesláním zprávy spočítá otisk této zprávy. Tento vypočítaný otisk zašifruje svým privátním klíčem a spolu s vlastní zprávou pošle příjemci. Příjemce vypočítá veřejným klíčem vysílače také otisk přijaté zprávy a porovná vypočítaný otisk s otiskem, který získal od příjemce. Pokud jsou oba otisky totožné, je tedy přijatá zpráva v takovém tvaru, v jakém ji vysílač skutečně poslal. Celý postup je vidět na obrázku:



Algoritmus samotný je založen na problému výpočtu diskretního logaritmu:

Nechť m, q, k, Y jsou přirozená čísla, pro něž platí $Y \equiv q^k \pmod{m}$.

Potom každé číslo k odpovídající uvedené rovnici nazveme diskretní logaritmus o základu q z Y vzhledem k modulu m . Tato definice nedefinuje číslo k jednoznačně, proto se někdy upravuje tak, že ze všech možných diskretních logaritmů ve smyslu předchozí definice se vybere ten nejmenší. Zatímco spočítat Y ze znalosti k, m, q je snadné, spočítat diskretní logaritmus k je velmi obtížné.

Vytváření klíčů

Vytváření klíčů má dvě fáze. Ve fázi první se vyberou parametry algoritmu, které mohou být sdíleny více různými uživateli systému.

- Vybere se výběr kryptografické hašovací funkce. V původní DSS byla jako hašovací funkce povinně SHA-1, ale v současných verzích je povolena též SHA-2.
- Dále se rozhodne o parametrech L a N , které určují délku klíče. V původní verzi DSS byla volba L omezena na násobky 64 v rozsahu 512 až 1024 včetně. NIST doporučuje délku 2048 (respektive 3072) pro klíče, u kterých se předpokládá používání po roce 2010 (respektive 2030), při použití adekvátně velkého N . Federální standard pro práci s informacemi doporučuje dvojice L a N (1024,160), (2048,224), (2048,256) a (3072,256).
- Dále se vybere N -bitové prvočíslo q . Délka N musí být alespoň taková, jako délka výstupu použité hašovací funkce.
- Dále se vybere L -bitové prvočíslo p takové, že $p-1$ je násobek q .
- Nakonec se vybere g jako takové číslo, jehož multiplikativní řád modulo p je právě q . Toho lze dosáhnout dosazováním do vzorce $g=h^{(p-1)/q} \bmod p$ pro náhodná h (kde $1 < h < p-1$), dokud výsledek není různý od jedné. Většina náhodných voleb h uspěje, nejčastěji se používá $h=2$.

Všechny výše zmíněné hodnoty mohou být sdíleny více uživateli a nejsou tajné. Následuje vytvoření samotných klíčů.

Nejdříve se náhodně vybere x v rozsahu $0 < x < q$.

pak se spočítá $y=g^x \bmod p$

Veřejný klíč je pak dán jako čtveřice (p, q, g, y) , soukromý klíč je dán jako x .

Podepisování

Při označení hašovací funkce písmenem H a zprávy písmenem z probíhá podepisování takto:

- pro danou zprávu se vybere náhodná hodnota k v rozsahu $0 < k < q$
- spočítá se $r = (g^k \bmod p) \bmod q$
- spočítá se $s = (k^{-1}(H(z) + x \cdot r)) \bmod q$
- v nepříliš pravděpodobném případě, že je $r=0$ nebo $s=0$ se výpočet opakuje od začátku
- jinak je podpisem dvojice (r, s)

Ověřování podpisu

- pokud neplatí $0 < r < q$ a $0 < s < q$ pak je podpis automaticky zamítnut.
- jinak se spočítá $w = (s)^{-1} \bmod q$
- dále se spočítá $u1 = (H(z) \cdot w) \bmod q$
- dále se spočítá $u2 = (r \cdot w) \bmod q$
- nakonec se spočítá $v = ((g^{u1} \cdot y^{u2}) \bmod p) \bmod q$
- Podpis platí, pokud platí $v = r$

Užití algoritmu

Digital Signature algorithm je široce využíván, mimo jiné v OpenSSL, v OpenSSH a v GnuPG.

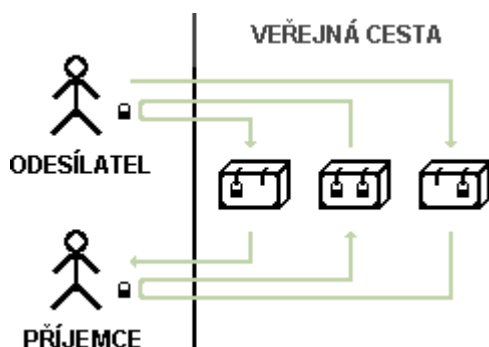
ElGamal

ElGamal je jeden z algoritmů asymetrické kryptografie, má ovšem nevýhodu, že šifrovaná data jsou dvakrát delší než data nešifrovaná. To je možná důvodem, proč není jeho nasazení tak masivní jako nasazení algoritmu RSA, který tímto nedostatkem netrpí. Opírá se o problém výpočtu diskretního logaritmu.

Diffie-Hellman

Je protokol výměny klíčů, který umožňuje navázat bezpečné spojení. Pro toto bezpečné spojení je nutné vyměnit si klíč k symetrické šifře přes ještě nezabezpečený kanál. Tento protokol se stará aby klíč nebyl poslán otevřeným kanálem.

Princip této metody si lze názorně popsat na příkladu s truhlou. Náš spojenec nemá klíč od visacího zámku, kterým jste truhlu zamkli. Vezme však svůj zámek a tím truhlu také zamkne – na truhle jsou nyní dva zámky. Takto zamčenou truhlu nám vrátí, my svůj zámek sundáme a pošlete mu ji zpět. On již jen odemkne svůj zámek a zprávu si přečte. Ke zprávě se celou dobu nemohl dostat nikdo cizí, truhla byla stále zamčena alespoň jedním zámkem:



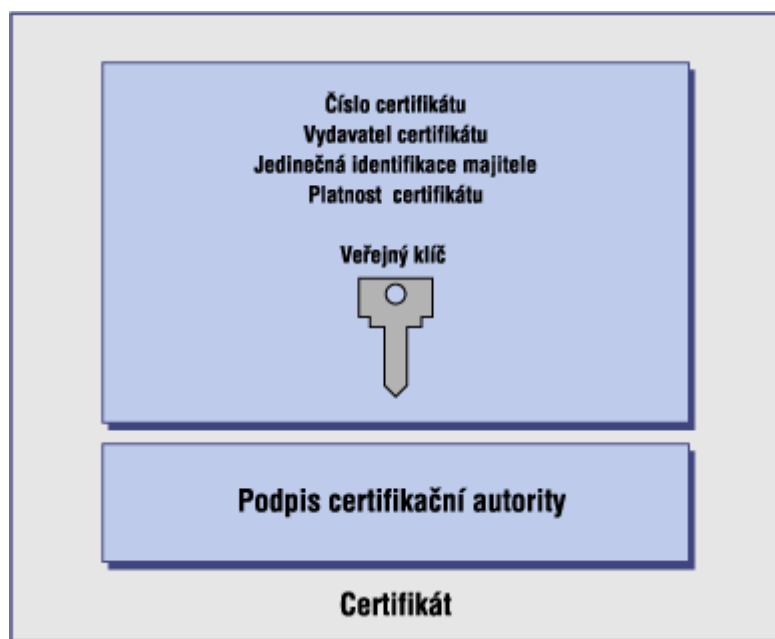
Bezpečnost Diffie-Hellmanova systému je také založena na obtížnosti řešení úlohy diskretního logaritmu. Tato metoda se nakonec příliš nerozšířila, neboť se objevila jiná, mnohem praktičtější metoda – RSA.

Certifikační autorita

Certifikační autorita je instituce, která se zabývá řešením distribuce a zprávy klíčů.

Certifikační autorita vystupuje při vzájemné komunikaci dvou subjektů jako třetí nezávislý důvěryhodný subjekt, který prostřednictvím jím vydaného certifikátu jednoznačně svazuje identifikaci subjektu s jeho dvojicí klíčů respektive s jeho digitálním podpisem.

Certifikáty obsahují ve své nejjednodušší formě veřejný klíč, jméno a další údaje zajišťující nezaměnitelnost subjektů. Běžně používané certifikáty též obsahují datum počátku platnosti, datum ukončení platnosti, jméno certifikační autority, která certifikát vydala, sériové číslo a některé další informace.



Vydáním certifikátu, je odstraněna nutnost důvěryhodné výměny klíčů mezi dvěma subjekty navzájem a jejich dohoda spočívá pouze v domluvě o společně uznávané Certifikační autoritě. Důležité je, že se utajovaná data na straně klienta redukuje pouze na bezpečné uchovávání privátního klíče, protože ostatní je řešeno certifikáty. Velkou výhodou certifikační autority je umožnění důvěryhodné komunikace mezi subjekty, které se nikdy fyzicky nepotkali ani neabsolvovali složitou výměnu veřejných klíčů.

V ČR jsou akreditovány tři certifikační autority:

1. Česká pošta s.p. - <http://www.postsignum.cz/>,

Ceník služeb: <http://vca.postsignum.cz/www/prices.php>

2. První certifikační autorita a.s. - <http://www.ica.cz/>

Ceník služeb: <http://www.ica.cz/cz/menu/5/cenik/>

3. Eidentity a.s. - <http://www.eidentity.cz/>

Ceník služeb: <https://www.eidentity.cz/Prices.html>